The Substrate Is the Bottleneck: A JEPA-style Code World Model on Code Edits and Program Execution

CodeLeWM Maintainers

Draft of May 31, 2026

Abstract

Joint-embedding predictive architectures (JEPAs) learn by predicting future latent states rather than reconstructing observations. They have been studied primarily in perception and control, where observations and actions are dense, continuous, and often smooth. We ask whether the same latent-transition recipe can learn useful dynamics over code. The answer depends less on the objective than on the substrate. We apply the same encoder, action encoder, predictor, SIGReg-style anti-collapse term, action-swap contrastive term, and inverse-action reconstruction term to two code substrates. The first substrate is commit-message-conditioned Python edits over CommitPackFT-style data. Across four scaled Hugging Face Jobs runs, it fails the action-conditioning gate: headline retrieval loses to the no-action baseline, mutation surprise is chance-level, and the v0.2 run collapses to an effective-rank ratio of 0.016. The second substrate is input-conditioned program execution: $(\text{code}, \text{input}) \mapsto \text{output}$ triples produced by a deterministic sandboxed Python executor. On the same architecture and objective registry, the v0.6 execution substrate produces non-collapsed, action-discriminative latents across two seeds: the no-action margin flips from -0.77 to +1.24, SIGReg drops by about 1200×, and the predicted-latent effective-rank ratio reaches 0.47. A downloaded-artifact eval pass adds a narrow positive downstream result: execution-pack retrieval beats no-action by +61.4 Recall@1 points and +65.9 MRR points on average across two seeds, and generated surprise decoys reach AUC 1.0 on all configured decoy categories. The result is partial, not a broad code-generation claim: latent probes remain blocked by lexical controls, crash prediction is not evaluable because the val/test slice has no positives, and HumanEval/MBPP-Plus reranking is not yet scored. The main lesson is that substrate signal-to-noise dominates objective tuning: pick the substrate before picking the architecture.

1 Introduction

JEPA-style world models replace pixel or token reconstruction with prediction in a learned latent space. In images and video, this idea has produced self-supervised representations that avoid direct generative decoding while still learning useful structure [1, 5]. LeWorldModel extends the same latent-prediction view to end-to-end visual world models trained from pixels and actions [13]. Code is an intentionally hostile domain for that recipe. It is discrete, structured, sparse, and semantically brittle: one token can change behavior while large textual edits can be inert. The question is not just whether a JEPA objective can be ported to code, but which code substrate gives the objective enough signal to learn anything.

We study this question through a controlled negative-to-positive comparison in CodeLeWM, a latent world model for Python code transitions. In both settings, the model predicts the latent of

an after-state from the latent of a before-state and an action:

$$\hat{z}_{\text{after}} = P(E(s_{\text{before}}), A(a)). \tag{1}$$

The state encoder E, action encoder A, predictor P, latent dimension, and objective terms are held fixed. Only the substrate changes.

Substrate A treats a transition as a commit-message-conditioned code edit: (before code, commit message) $\mapsto$ after code. It is attractive because it resembles the product surface of a patch scorer, but it has three structural problems. First, after $\approx$ before is often a strong prior. Second, commit messages are noisy, underspecified actions. Third, commits are often multi-purpose, so no single canonical action-conditioned transformation exists. The scaled evidence confirms the diagnosis. Four public runs all fail the headline retrieval comparison against no-action; the strongest v0.2 run reports an effective-rank ratio of 0.016, chance-level mutation surprise AUC 0.501, and latent probes beaten by lexical or metadata controls.

Substrate B keeps the architecture and objective unchanged but changes the world. It treats a transition as a deterministic execution triple: (code, input) $\mapsto$ output. Here the output is not a near-copy of the code, the input is a precise typed action, and each row is single-purpose. The v0.6 execution-substrate run inverts the failures above: two training seeds reach effective-rank ratios of 0.467 and 0.477, the no-action margin becomes strongly positive, and the execution-pack retrieval gate beats no-action by more than 60 absolute Recall@1 points.

The contribution is deliberately scoped:

1. A reproducible JEPA-style latent-transition recipe for code transitions, with manifest-backed data, checkpoints, and eval reports.

2. A controlled negative result on commit-message-conditioned code edits, showing that objective tweaks do not rescue a low-signal substrate.

3. A partial-positive execution-substrate result under the same architecture: non-collapse, action use, execution-pack retrieval, and generated-decoy surprise pass across two seeds.

4. An explicit claim boundary: the result is not a code generator, not a general semantic-axis result, and not yet HumanEval/MBPP-Plus reranking evidence.

2 Related Work

Joint-embedding predictive architectures. I-JEPA predicts target image-block embeddings from context embeddings without pixel reconstruction [1]. V-JEPA extends this idea to video feature prediction [5]. LeWorldModel uses a latent prediction model for action-conditioned visual dynamics from pixels [13]. Our work preserves the same core contract but changes the domain to code and compares two substrates under a fixed objective.

Anti-collapse regularization. The central technical risk in latent predictive models is collapse: a constant encoder can make prediction losses appear small. VICReg and Barlow Twins add variance, covariance, and redundancy-reduction constraints to self-supervised learning [4, 16]. LeJEPA and LeWorldModel study SIGReg-style regularization for stable joint-embedding learning [3, 13]. CodeLeWM inherits that anti-collapse emphasis and makes collapse gates a first-class artifact.

Code representations and code generation benchmarks. Pretrained code models such as CodeBERT, GraphCodeBERT, UniXcoder, and CodeT5 learn representations or generation models from program text and natural language [8–10, 15]. CodeNet, MBPP, APPS, HumanEval, and EvalPlus/MBPP-Plus provide benchmark substrates for program understanding, synthesis, and evaluation [2, 7, 11, 12, 14]. CodeT uses generated tests to rerank code generations [6]. CodeLeWM is not a generator: it is a local latent scorer/reranker that can be applied to candidate code or execution completions once labeled candidate artifacts exist.

3 Model and Objective

Both substrates use the same transition model. A state encoder E maps a bounded token capsule to $z \in \mathbb{R}^{256}$. An action encoder A maps either a commit message or an input representation to an action embedding. A predictor P maps $(z_{\text{before}}, A(a))$ to $\hat{z}_{\text{after}}$. The training loss is

$$\mathcal{L} = \|P(E(s_b), A(a)) - E(s_a)\|_2^2 + \lambda_{\text{sig}}\mathcal{L}_{\text{SIGReg}} + \lambda_{\text{swap}}\mathcal{L}_{\text{swap}} + \lambda_{\text{inv}}\mathcal{L}_{\text{inv}}. \tag{2}$$

The prediction term optimizes latent transition accuracy. SIGReg discourages collapse. The action-swap term penalizes predictions under mismatched actions, and the inverse-action term asks the model to preserve action information in the latent transition. The v0.2 and v0.6 runs use the same objective family; the comparison is therefore about the observation/action substrate, not a new architecture.

4 Substrates

Substrate A: commit-message-conditioned edits. Substrate A uses permissively licensed CommitPackFT-style Python transitions. The before-state and after-state are code capsules; the action is a natural-language commit message. The v0.2 action-swap/inverse-action run packs 20,645 transitions with train/val/test counts 18,019/1,291/1,335. Its public report is the v0.2 action-swap HF results report; the run artifact id is listed in the claim audit.

Substrate B: input-conditioned execution. Substrate B uses deterministic Python execution records. The before-state is code, the action is a serialized input, and the after-state is a serialized output. The execution pack contains 1,605 records from CodeNet, MBPP, and APPS, with HumanEval and MBPP-Plus held out for downstream evaluation. Candidate source code is treated as untrusted input; sandbox execution is a data-build or operator-labeling step, not a model-training or model-scoring step. The v0.6 pack id is `codelewm-execution-pack-20260528T102625Z`; the report is `docs/benchmark/EXECUTION_V0_6_RESULTS_2026-05-30.md`.

5 Experimental Protocol

All quantitative claims in this draft trace to checked-in benchmark reports or schema-versioned artifacts. For Substrate A, we use the v0.2 downloaded-artifact verification report. For Substrate B, we use two HF Jobs training seeds, 42 and 1729, each trained to 50k steps on `a10g-small`. The #305 eval pass reruns the downloaded checkpoints through four JSONL execution-pack eval surfaces: execution retrieval, generated-decoy surprise, latent probes, and crash prediction. The committed eval tree is `docs/benchmark/v0_6/`; the same tree is mirrored to the HF dataset repo at the commit recorded in the claim audit.

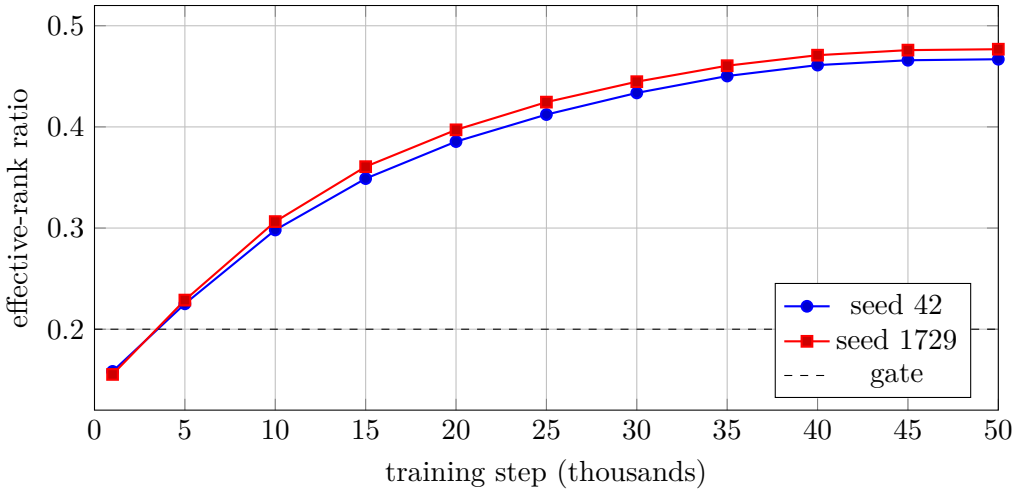

Figure 1: v0.6 predicted-latent effective-rank ratio from the collapse diagnostics report. Both seeds cross the 0.20 collapse gate early and finish near 0.47.

Table 1: Headline training and collapse metrics from the v0.2 and v0.6 benchmark reports.

Run	prediction MSE	SIGReg	no-action margin	effective-rank ratio
v0.2 action-swap	0.1494	n/a	n/a	0.0158
v0.6 seed 42	0.00064	0.0364	+1.2308	0.4669
v0.6 seed 1729	0.00060	0.0348	+1.2434	0.4768

6 Results

6.1 Training dynamics and non-collapse

The training-time substrate-pivot claim passes on Substrate B and fails on Substrate A. Substrate A v0.2 reaches an effective-rank ratio of only 0.016. Substrate B reaches 0.467 and 0.477 on seeds 42 and 1729, above the configured 0.20 gate. Figure 1 shows the v0.6 predicted-latent effective-rank ratio over training.

The v0.6 no-action margin also flips sign quickly and remains positive in raw training metrics (Figure 2). The report-level final metrics average to +1.2308 for seed 42 and +1.2434 for seed 1729.

6.2 Retrieval

Retrieval is the cleanest action-use measurement. For Substrate A, the model must rank true after-states above hard negatives, but no-action remains stronger. For Substrate B, the query is an execution record and the candidate set is 236 val/test outputs. Table 2 shows the contrast.

Mean v0.6 Recall@1 is 0.6525 and mean MRR is 0.7628. Mean lift over no-action is +0.6144 Recall@1 and +0.6588 MRR, with seed-to-seed spread below one point. This supports a narrow execution-pack retrieval claim.

6.3 Surprise and decoys

Substrate A v0.2 mutation surprise is chance-level: mutation AUC 0.501. Substrate B v0.6 generated-decoy surprise passes all configured numeric gates (Table 3). The caveat is important:

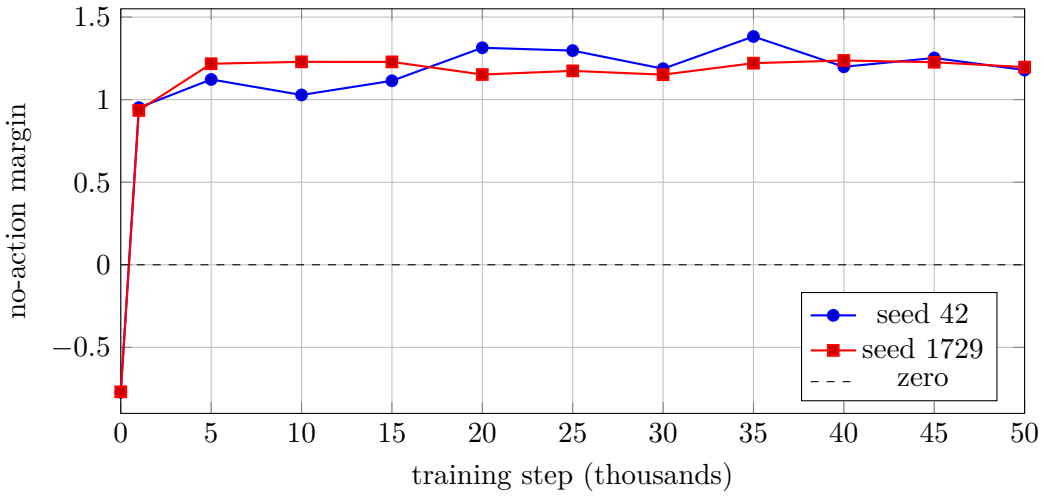

Figure 2: Raw v0.6 no-action margin training metric. Positive values mean the action-conditioned prediction is closer to the target than the no-action latent.

Table 2: Retrieval comparison. Positive deltas mean the action-conditioned model beats no-action.

Substrate	Run	Recall@1	Δ vs no-action	MRR	Δ vs no-action
A	#138	0.371	-0.088	0.473	-0.073
A	#154	0.363	-0.106	0.468	-0.082
A	#159	0.597	-0.053	0.675	-0.034
A	#172	0.263	-0.178	0.370	-0.163
B	v0.6 seed 42	0.657	+0.619	0.767	+0.663
B	v0.6 seed 1729	0.648	+0.610	0.759	+0.655

same-problem-different submission has only six generated pairs after filtering, so it is a positive diagnostic, not broad semantic surprise evidence.

6.4 Latent probes, crash prediction, and reranking

The broader downstream story is not positive. Only `output_type` has train/val/test labels for v0.6 latent probes. The latent view beats no-action by 5.7 points on both seeds, but lexical controls remain stronger (Table 4). The report therefore sets `positive_representation_claim_allowed=false`.

Crash prediction is not evaluable because both v0.6 val/test slices have 0 positives and 236 negatives. HumanEval/MBPP-Plus reranking is also not scored: the sampler and `codelewm.eval.completion_label.v` contract exist, but full live labeled-completion artifacts are not present. No HumanEval/MBPP-Plus pass@1 lift claim is allowed from the current evidence.

7 Discussion

The controlled comparison supports one main explanation: the substrate is the bottleneck. On commit-message-conditioned edits, the same architecture has a strong shortcut. A no-action or identity-like prior is often correct, and the action text is too vague to force a different latent prediction. Objective variants improve some diagnostics but do not invert the headline gate.

Table 3: v0.6 generated-decoy surprise AUCs from the committed per-seed surprise reports.

Run	mutation	same-code/different-input	same-problem/different-submission
v0.6 seed 42	1.000 / 236	1.000 / 195	1.000 / 6
v0.6 seed 1729	1.000 / 236	1.000 / 195	1.000 / 6

Table 4: v0.6 output-type latent-probe test accuracy.

Run	z_{pred}	no-action	lexical	claim
v0.6 seed 42	0.497	0.439	0.662	blocked
v0.6 seed 1729	0.599	0.541	0.618	blocked

Execution triples remove that shortcut. The output token distribution is different from the code distribution; copying the code latent is structurally wrong. The action is a deterministic value rather than a free-form commit message. Each row is single-purpose and judge-verifiable. Under these conditions, the same objective produces non-collapsed latents and strong execution-pack retrieval.

This does not mean the model understands programs in a broad sense. The probe result is deliberately humbling: lexical features still beat the latent view on the only evaluable semantic target. The surprise result is also limited by generated decoys. The right conclusion is not that CodeLeWM solves code generation; it is that a JEPA-style latent transition model can learn an action-conditioned execution substrate when the substrate supplies precise state/action/next-state signal.

8 Limitations

- The execution substrate is Python-only and stdlib-only under the current sandbox policy.
- The v0.6 result uses two training seeds. This is enough to check a large effect with small spread, but it is not a full variance study.
- HumanEval and MBPP-Plus reranking are not evaluated in the current artifact set. A live OpenRouter sampling and hidden-test labeling run is still required.
- Generated surprise decoys are useful diagnostics but not adversarial semantic tests. The same-problem decoy category has only six pairs.
- The paper compares substrates inside one implementation and objective registry. It does not claim superiority over large pretrained code models.

9 Reproducibility and Artifacts

The public artifact chain is intentionally part of the result. Substrate A uses `abdelstark/codelewm-public-shard`, `abdelstark/codelewm-transition-model`, and `abdelstark/codelewm-runs`. Substrate B uses `abdelstark/codelewm-execution-pack@v0.6.0` and the two seed-specific v0.6 run artifacts in `abdelstark/codelewm-runs`. The #305 eval reports are committed under `docs/benchmark/v0_6/` and mirrored to the HF dataset repo. Every published artifact is manifest-backed and secret-scanned.

The claim audit for this draft is `docs/papers/two_substrate_claim_audit.md`. The build command is:

```
scripts/build-two-substrate-paper
```

10 Conclusion

The negative v0.2 result and partial-positive v0.6 result are not contradictory. They are the point. A JEPA-style code world model does not become useful merely because the objective includes prediction, anti-collapse regularization, and action-contrastive terms. It needs a substrate where the action actually constrains the next state. For code, commit-message-conditioned edits did not provide that signal; deterministic execution triples did. The practical lesson for future code-world-model work is simple: pick the substrate before picking the architecture.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL https://arxiv.org/abs/2301.08243.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

[3] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL https://arxiv.org/abs/2511.08544.

[4] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. URL https://arxiv.org/abs/2105.04906.

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL https://arxiv.org/abs/2404.08471.

[6] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. URL https://arxiv.org/abs/2207.10397.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

[8] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages, 2020. URL https://arxiv.org/abs/2002.08155.

[9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow, 2021. URL https://arxiv.org/abs/2009.08366.

[10] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation, 2022. URL https://arxiv.org/abs/2203.03850.

[11] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps, 2021. URL https://arxiv.org/abs/2105.09938.

[12] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. URL https://arxiv.org/abs/2305.01210.

[13] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL https://arxiv.org/abs/2603.19312.

[14] Ruchir Puri, David S. Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, Veronika Thost, Luca Buratti, Saurabh Pujar, Shyam Ramji, Ulrich Finkler, Susan Malaika, and Frederick Reiss. Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks, 2021. URL https://arxiv.org/abs/2105.12655.

[15] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021. URL https://arxiv.org/abs/2109.00859.

[16] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow twins: Self-supervised learning via redundancy reduction, 2021. URL https://arxiv.org/abs/2103.03230.